

GeMPa: Generalizable Multi-Robot Patrolling with Recurrent Attention Policies

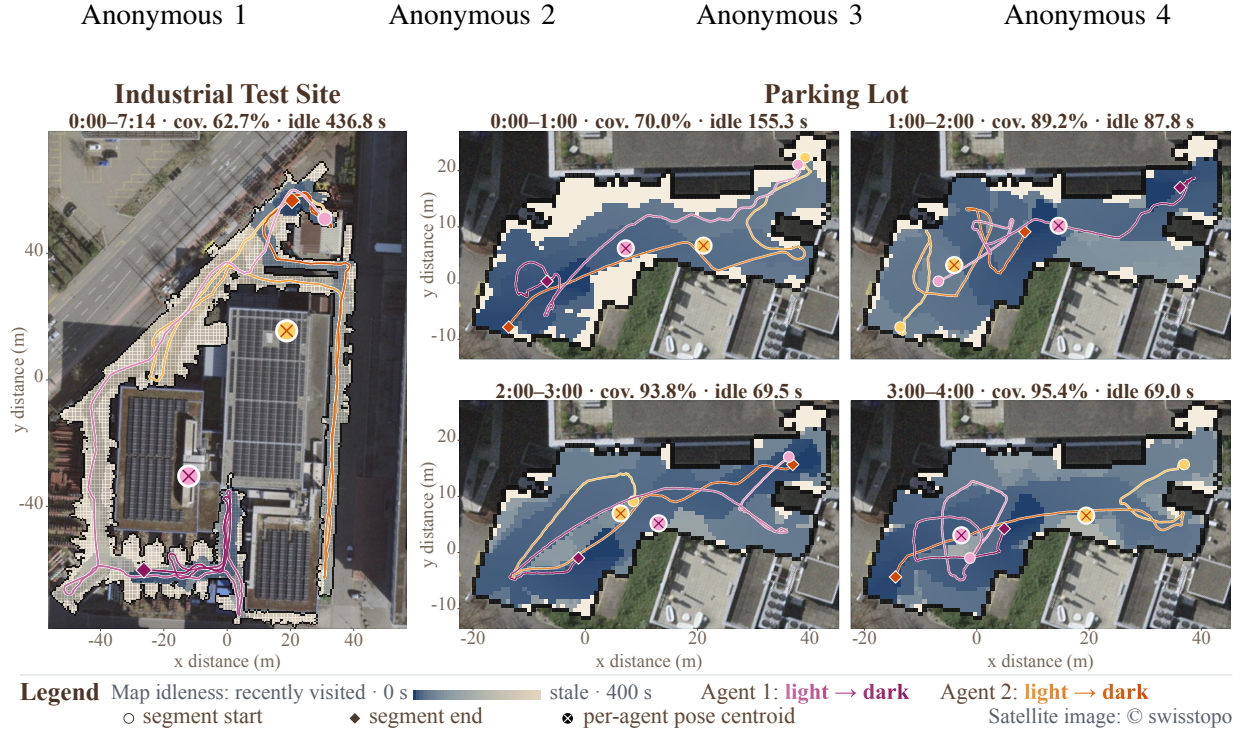


Fig. 1. GeMPa deployed in two real sites. A single policy, trained entirely in simulation and deployed without a patrol graph or any site-specific tuning, drives two Unitree Go2 quadrupeds through previously unseen outdoor environments. Left: a seven-minute patrol of a large industrial test site. Right: four consecutive one-minute segments of a parking lot patrol, showing the idleness map as it evolves; the team pushes coverage from 70.0% to 95.4% and mean idleness from 155.3s to 69.0s. Colour encodes cell idleness, dark blue for recently visited and pale for stale, and each robot’s trace darkens with time.

Abstract—Persistent multi-robot patrolling is a complex real-world problem, requiring repeated observation of an environment while adapting to changes in layout and team composition, under three requirements: *persistence*, *adaptability*, and *unpredictability*. We present GeMPa, a multi-agent reinforcement learning policy for visibility-based persistent monitoring that commands body-frame velocities, with no patrol graph, task allocation or planner in the control loop. It combines a compact egocentric idleness summary and lidar ranges with attention over a variable number of teammates and recurrent temporal reasoning. Trained with proximal policy optimization on procedurally generated map layouts with one to four robots, a single shared policy transfers zero-shot to unseen maps and teams of up to sixteen robots. It attains the lowest mean idleness at every team size, with a mean per-map idleness reduction of 45.1% relative to the next best method and idleness falls 91.5% from one to sixteen robots. Under map changes, robot losses, and communication delays at sixteen robots, its idleness increase is 85%, 89% and 76% smaller than the best baseline, while revisit-time variability remains comparable. The same weights run unmodified on simulated and physical Go2 quadrupeds at three sites, the largest eight times the average training map, matching simulated counterpart coverage to within 0.6 percentage points. Further materials: <https://paperanonyms.github.io/gempa/>.

Index Terms—Multi-Robot Systems, Cooperating Robots, Reinforcement Learning.

I. INTRODUCTION

Robotic systems are transitioning from controlled laboratory environments into real-world applications that require persistent autonomous operation. One such application is autonomous patrolling, where mobile robots can repeatedly monitor large environments. Unlike conventional coverage where the task terminates once the environment has been observed, persistent monitoring requires robots to continually revisit regions so no area remains unobserved for long periods of time. This objective is commonly quantified through the *idleness* or *staleness* of locations in the environment [1]–[3].

Human patrols already perform this task in the real world, with the doctrine developed for military operations being an established reference. Specifically, military guidance emphasizes **continuous reconnaissance** to maintain situational awareness and provide early warning [4], **avoidance of predictable patrol patterns** that could be exploited by an adversary [5] and **adaptation of plans as new information becomes available** [6]. Hence, these three requirements translate into the properties an autonomous robot patrol must satisfy: *persistence*, *unpredictability* and *adaptability*.

Existing formulations impose structural assumptions that

limit adaptability. Geometric visibility methods assume a known polygonal environment and terminate once every point has been observed [7], [8]. Classical patrolling keeps the recurring objective but abstracts the environment into a graph or target set in which observation means arrival at a node [1], [2], [9], and persistent monitoring adds a model of how information decays between visits while still planning over those same nodes [10]. Visibility-based formulations remove that abstraction, crediting observation to the free cells inside the sensor footprint, and multi-agent reinforcement learning has been applied to them [3], [11]. Those agents, however, move one cell at a time on a small discrete grid and receive the exact floor plan at both training and deployment; neither work evaluates layouts differing in topology or scale from training, and hardware validation in [11] is a laboratory replica of the training grid, so neither demonstrates persistence beyond the training site or adaptability to an unseen one.

We therefore present GeMPa, a multi-agent reinforcement learning policy for visibility-based persistent monitoring with a variable team, formulated so that one set of weights is deployable across sites and team sizes without retraining. GeMPa is a shared recurrent policy trained with Multi-Agent Proximal Policy Optimization (MAPPO) that pools a variable number of teammate tokens by cross-attention, fuses them with an egocentric idleness summary and lidar ranges through two GRU layers, and outputs velocity commands. Each choice removes a dependence: the velocity interface transfers behaviour to a physical platform without an intermediate grid, the fixed sector count makes input width independent of site size, range sensing lets the policy react to obstacles it was never trained on, and the masked attention removes input assumptions of a fixed team size. We train one policy on procedurally generated floor plans with one to four robots under centralised training with decentralised execution [12], [13], then evaluate it zero-shot on held-out layouts and team sizes.

We show that the model generalizes in the real world on unseen floor plans, up to 8 times the size of the average training map and with teams up to 4 times larger than any seen in training. GeMPa achieves 91.5% idleness decrease from one to sixteen robots while maintaining revisit-time variability comparable to MA-G-PPO and up to 89% better robustness to disturbances, compared to the best baseline. Our contributions are:

- 1) Combining attention-based multi-agent information exchange with recurrent temporal reasoning to support persistent and adaptive multi-robot patrolling.
- 2) Demonstrating zero-shot generalization to previously unseen environments and team sizes, substantially larger than the training set.
- 3) Deploying the same learned weights on physical quadruped robots and evaluating persistent-monitoring performance in real environments.

II. RELATED WORK

a) Visibility coverage: Geometric visibility coverage is classically studied through the Art Gallery Problem (AGP) [7],

which seeks a minimum set of stationary guards whose combined visibility covers a polygonal environment. Its mobile counterpart, the Watchman Route Problem (WRP) [8], [14], seeks a shortest route from which the entire environment can be observed. Both problems are NP-hard and the hardness persists for observers with limited field of view and sensing range [15]–[19]. Also, they target a single complete observation of a static environment: the route is computed offline, is not revised when the layout changes, and, being deterministic, is fully predictable. Autonomous Coordinated patrolling instead requires a team to re-observe free space as information goes stale, and to avoid obstacles and stay effective when the layout changes.

b) Persistent monitoring: Persistent monitoring and Patrolling address repeated observation by modeling how information quality or environmental uncertainty evolves between visits. In multi-robot patrolling, this is commonly expressed through node *idleness*, evaluated using average or worst-case revisit intervals [1], [2], [9]. Existing approaches include optimal control of uncertainty dynamics [20], [21], speed regulation along prescribed paths [22], cooperative trajectory optimization [23], and receding-horizon planning [24]. These approaches plan over a fixed set of nodes or paths and assume a visit observes the whole region a node represents. Two consequences follow: the policy cannot reroute when the environment changes, and a fixed waypoint set makes the patrol easy to anticipate. Guo et al. [25] address the second by trading efficiency for route entropy, but the graph remains fixed, which caps both unpredictability and adaptability.

c) Visibility-based persistent monitoring and learned policies: Visibility-based persistent monitoring credits coverage to the cells inside the sensor footprint rather than to visited nodes [3]. MA-G-PPO [3] encodes global and local idleness maps convolutionally and exchanges them by graph attention, studying generalisation across agent counts in a simplified environment. Like most work in this line, it assumes a square omnidirectional footprint, unlike a legged robot whose forward-facing camera must be swept. GALOPP [11] demonstrated learned persistent monitoring with aerial robots in a protected environment, to our knowledge the first real-world deployment of a learned multi-agent monitoring policy, but likewise assumes full knowledge of a simplified map. GeMPa instead pairs a fixed-width egocentric idleness summary with masked teammate cross-attention and recurrent integration, and evaluates one set of weights, without fine-tuning, across unseen layouts, map scales, team sizes, and mid-episode layout changes, validated in real-world deployments.

III. METHOD

a) Problem formulation: We formulate the problem as a multi-agent partially observable Markov game involving a team of robots $\mathcal{N} = \{1, \dots, N\}$; we use agent and robot interchangeably, preferring the former in the learning formulation and the latter for the physical platform. The agents repeatedly observe free space to maintain low idleness, defined by the time elapsed since each location was last

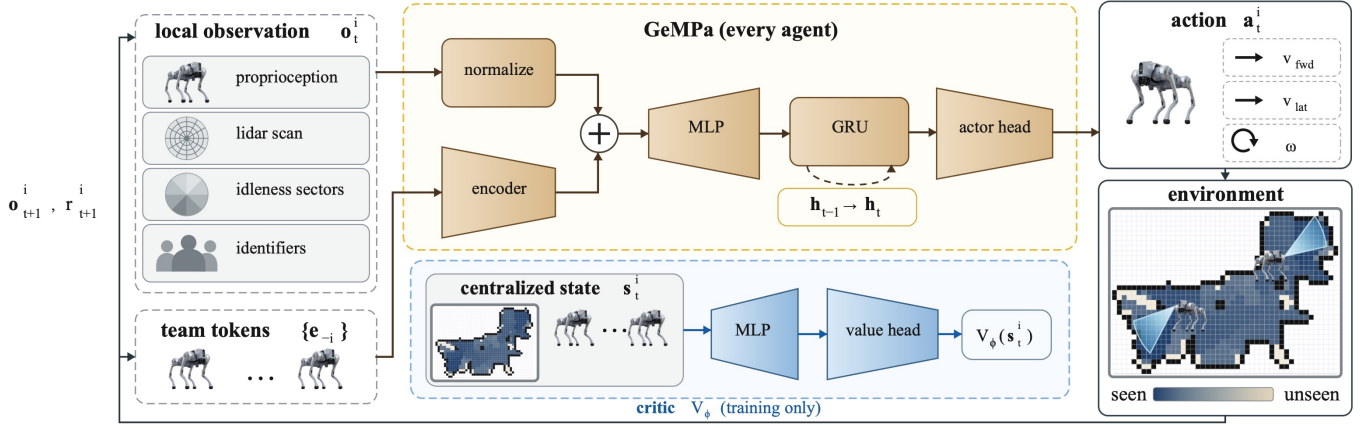


Fig. 2. **GeMPa architecture.** Every robot runs the same recurrent actor π_θ . The local observation, comprising position, heading, a 16-ray lidar scan, an 8-sector egocentric idleness histogram and the agent identifiers, is encoded alongside a variable-size set of teammate tokens, one per active robot, which cross-attention pools into a fixed-width team embedding that is invariant to team size and ordering. An MLP fuses the two streams and a GRU carries the state needed to avoid recently visited regions, yielding factored velocity commands (v_x, v_y, ω) . A centralized critic V_ϕ sees global team and map information during training only and is discarded at deployment, so action selection is fully decentralized. The environment panel shows the evolving exploration state, with visited regions in blue, unvisited free space in beige and occupied cells in white.

observed. The environment state s_t evolves according to the joint action $\mathbf{a}_t = (a_t^1, \dots, a_t^N)$, while each agent receives only a partial observation $\mathbf{o}_t^i$ of the underlying state s_t . Each robot seeks to maximize its expected discounted return, $J_i(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t r_t^i \right]$, where π_θ denotes the joint policy, γ is the discount factor, and T is the rollout horizon. The reward combines a shared monitoring objective with agent-specific motion penalties.

b) Simulation environment: GeMPa acts as a high-level planner sending velocity commands to the robot’s low-level motion controller. Training therefore does not require detailed robot dynamics or photorealistic 3D rendering. We use a lightweight simulator that exposes only the task-relevant state, i.e. agent poses, cell idleness, and field of view (FoV), speeding up the training. The environment is a planar occupancy grid $\mathcal{K}$ with cell size δ , partitioned into free cells $\mathcal{F} \subset \mathcal{K}$ and obstacle cells, simulated in a custom Python environment. Agent i observes the FoV $\mathcal{V}_t^{(i)} \subset \mathcal{F}$, the set of free cells whose centers lie within range ρ of the agent and within $\pm\alpha$ of its heading. Motion blur is modeled by shrinking the effective range linearly with yaw rate, so that a rapidly rotating agent perceives fewer cells than a slowly turning one. Each free cell $k \in \mathcal{F}$ carries an age $g_k(t)$ representing the number of steps since it was last observed, and it is reset to zero whenever the cell enters any agent’s FoV.

c) Reward: The reward combines the persistent monitoring objective with the constraints of physical deployment:

$$r_t^{(i)} = -w_I \bar{I}_t - w_c c_t^{(i)} - w_h h_t^{(i)} - w_a \mathcal{K}_t^{(i)} - w_s \left\| \mathbf{a}_t^{(i)} - \mathbf{a}_{t-1}^{(i)} \right\|_2 \quad (1)$$

The first term of Eq. 1 is the monitoring objective and drives the team to minimize the time since each cell was last observed. The environment tracks this quantity as the age $g_k(t)$ of each free cell, which is unbounded and therefore unsuitable as a reward [12]. We instead define the idleness

$I_k(t) = 1 - e^{-g_k(t)/\tau}$ of cell k , which saturates at one as the cell grows stale and is pinned at $I_k = 1$ for unseen cells, and take the team objective to be its mean over the free cells:

$$\bar{I}_t = \frac{1}{|\mathcal{F}|} \sum_{k \in \mathcal{F}} I_k(t) \quad (2)$$

The remaining terms encode deployment constraints. The second penalises wall contact through the fraction of commanded velocity lost against an obstacle $c_t^{(i)} \in [0, 1]$. The third enforces clearance through the squared hinge $h_t^{(i)} = \max(0, 1 - d_t^{(i)}/d_m)^2$, where $d_t^{(i)}$ is the distance to the nearest obstacle and d_m a safety margin. The fourth term charges a fixed cost whenever the indicator $\mathcal{K}_t^{(i)}$ signals contact with a teammate. These three terms yield safe navigation among static obstacles and other agents. The final term penalises changes between consecutive commands and regularises the policy towards smooth trajectories that a physical platform can track.

d) Observation and action spaces: We construct a compact observation space that captures monitoring and navigation information by summarizing map idleness as a fixed-size vector in the robot’s frame. Each robot receives a local observation and a variable-size set of team tokens. The local observation at timestep t for agent i is:

$$\mathbf{o}_t^i = [\mathbf{p}, \boldsymbol{\psi}, \boldsymbol{\ell}, \boldsymbol{\iota}, \boldsymbol{\eta}] \quad (3)$$

where $\mathbf{p} \in \mathbb{R}^2$ is the ego position normalized by the map half-extents and $\boldsymbol{\psi} = [\sin(\theta), \cos(\theta)] \in \mathbb{R}^2$ the sine-cosine encoding of the ego heading. $\boldsymbol{\ell} \in \mathbb{R}^{16}$ contains body-frame lidar ranges normalized by the 8 m sensor range; it gives the policy the local geometry needs for obstacle avoidance and the shape of its neighborhood. $\boldsymbol{\iota} \in \mathbb{R}^8$ is an egocentric idleness histogram: its k -th entry is the mean idleness of the free cells whose bearing from the ego agent falls in the k -th of eight 45° body-frame sectors, which the policy uses to move towards regions of high idleness. Finally, $\boldsymbol{\eta} \in \mathbb{R}^2$ holds the normalized

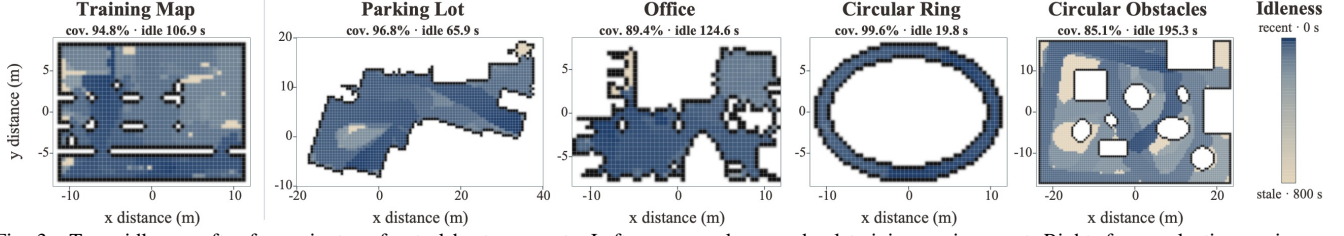


Fig. 3. Team idleness after four minutes of patrol by two agents. Left: an example procedural training environment. Right: four evaluation environments, comprising two real floor plans and two synthetic layouts. Color indicates time since the most recent visit by either agent, from recent (dark blue) to stale (beige); white regions are obstacles or outside the traversable area. Maps are scaled independently, with metric axes retained for size reference.

agent index and active team size, which let the shared policy adopt a role consistent with the number of teammates it patrols with.

Teammates are summarized by team tokens $\{e_{-i}\}$, where each token $e_j \in \{e_{-i}\}$ is defined as:

$$e_j = [d^{(j)}, \beta^{(j)}, \Delta\psi^{(j)}, \kappa^{(j)}] \quad (4)$$

where $d^{(j)} \in \mathbb{R}$ is the geodesic (obstacle-aware) distance to agent j normalized by the map half-extent, $\beta^{(j)} \in \mathbb{R}^2$ is the sine and cosine of its bearing in the ego frame, $\Delta\psi^{(j)} \in \mathbb{R}^2$ encodes its heading relative to the ego agent, and $\kappa^{(j)} \in \mathbb{R}$ is its normalized slot index.

During training, a separate critic estimates a per-agent value from a centralized input that augments the agent’s own observation with privileged information unavailable at execution:

$$s_t^i = [\mathbf{o}_t^i, \bar{\mathbf{e}}, \mathbf{e}^{\max}, \bar{d}, \bar{l}, l^{\max}] \quad (5)$$

where $\bar{\mathbf{e}}, \mathbf{e}^{\max} \in \mathbb{R}^7$ are the element-wise mean and maximum over privileged teammate tokens, which replace the slot index $\kappa^{(j)}$ of e_j with the relative velocity $\nu^{(j)} \in \mathbb{R}^2$ of agent j in the ego frame and are computed from true rather than estimated poses; $\bar{d}$ is the mean pairwise geodesic distance between active agents; and $\bar{l}, l^{\max}$ are the mean and maximum idleness over the free cells of the map. Pooling by mean and maximum collapses any number of teammates to a fixed-size summary, and is zeroed when the agent patrols alone. All privileged terms enter only the critic, so the executed policy requires no privileged state.

The policy outputs body-frame velocity commands, providing the same command interface in simulation and deployment while delegating motion execution to the robot’s low-level controller. This makes the policy portable to platforms capable of tracking these commands. The action is $\mathbf{a}_t = [v_x, v_y, \omega] \in \mathbb{R}^3$, where v_x and v_y are the forward and lateral linear velocities and ω is the yaw rate.

e) Model architecture: We train GeMPa as a shared recurrent actor and a centralized critic, using Multi-Agent Proximal Policy Optimization (MAPPO) [12], under centralized training with decentralized policy execution. All robots share actor parameters but maintain separate recurrent states.

As shown in Fig. 2, the actor encodes the variable-size set of teammate tokens $\{e_{-i}\}$ using a cross-attention encoder [26]. Four learned latent queries attend to the tokens through

four attention heads of dimension 32, producing four latent outputs that are concatenated and linearly projected to a 64-dimensional team embedding. Absent teammate tokens are excluded by an attention mask, allowing the same encoder to accommodate different team sizes.

The team embedding is concatenated with the ego features $\mathbf{o}_t^i$, passed through a fully connected layer of 128 units, and processed by two Gated Recurrent Unit (GRU) layers with 128 hidden units each. GRUs were used instead of LSTMs, following common practice in the MARL literature [12]. The GRU receives these fused features together with the agent’s previous recurrent state, integrating observation history to provide temporal context under partial observability. An action head produces five outputs per velocity axis, parameterizing discrete distributions over forward, lateral, and yaw velocity commands.

The centralized critic is a feed-forward network with two fully connected layers of 128 units. It does not use the encoder’s information, but receives privileged information, s_t^i .

During MAPPO training, the critic is trained from each agent’s individual reward sequence, and its value estimates are used to compute advantages for the actor update. At deployment, the critic is discarded: each agent selects actions using only its ego features, available teammate information and recurrent memory.

f) Domain randomization: To facilitate the deployment of GeMPa to the real world, we introduce variation in the training environments and perturb the motion and observation models. To model imperfect velocity tracking, the applied velocity follows the command through a first-order lag, $\mathbf{v} \leftarrow \mathbf{v} + (1 - e^{-\Delta t/\tau_v})(\mathbf{a}_t - \mathbf{v})$. Each axis of the applied velocity is then perturbed by multiplicative Gaussian noise, $\mathbf{v} \leftarrow \mathbf{v}(1 + 0.1\epsilon)$ with $\epsilon \sim \mathcal{N}(0, 1)$, so that tracking errors accumulate into position and heading drift over the episode. Each lidar range is scaled by multiplicative Gaussian noise of 15%, the largest level we experimentally found to not degrade performance. Independently, each ray is dropped to its maximum range with probability 0.05 at every time step.

g) Training setup: Training is conducted on 1,000 procedurally generated indoor layouts of 18×24 m, which differ in corridor topology, room partitioning, door placement, and randomly placed free-standing obstacles; an example is shown in Fig. 3. The number of layouts is chosen to expose the agents to a broad distribution of environments and thus promote

generalization to unseen maps, while the map size trades off training speed against an environment large enough for the policy to learn non-trivial patrolling behavior and transfer to maps of different scale. Every episode samples a new layout, episodes last 400 s and training runs for 25 million agent-steps. Each training run takes approximately 1.9 h of wall-clock time on a single NVIDIA L4 GPU.

IV. EXPERIMENTAL EVALUATION

Motivated by the requirements of persistence, adaptability and unpredictability, we evaluate whether GeMPa maintains fresh observations, responds effectively to changes during a patrol, and retains variability in revisit timing. The latter measures temporal irregularity rather than an observer’s ability to predict patrol trajectories. We compare GeMPa against one learned and two map-informed baselines on unseen maps, unseen team sizes and disturbances applied at inference. Matched simulation and hardware runs assess sim-to-real transfer using the Go2 quadruped, and ablations examine the contributions of temporal memory and idleness-sector observations.

a) Evaluation protocol: Evaluation uses 14 held-out maps whose free areas range from $0.15\times$ to $4.5\times$ those of the training maps. Six are real-world sites (including indoor and outdoor areas with varying sizes and layouts); the remaining eight are synthetic and split into high-collaboration layouts, whose topology requires the agents to coordinate, and easy layouts with large open spaces, closer to the training distribution, where coordination matters less. Examples of these maps and the resulting idleness fields at the latest time step are shown in Fig. 3. Each map is evaluated with $N \in \{1, 2, 4, 8, 16\}$ robots, covering team sizes within the training range and teams two and four times larger than the maximum used during training. Each policy is evaluated on episodes of length 800 s (twice the training episode duration) on all 14 held-out maps, for 5 seeds, 5 team sizes, and 20 randomly sampled initial configurations per map and team size, for a total of 7000 evaluation episodes per method.

To compare under common sensing and motion constraints, we adapt MA-G-PPO’s field of view to our visibility model and replace its action interface with the same discretized body-frame velocity commands. It is trained for 25 million agent-steps on the same maps, spawning positions and varying team sizes, while retaining its image-based representation, graph-attention architecture and decentralized critic. Thus, MA-G-PPO denotes the adapted baseline throughout this section. We further select GCS [3] and Watchman [8], two map-informed planner baselines. GCS is the centralized greedy planner reported alongside MA-G-PPO: it repeatedly assigns each robot a shortest path to a nearby stale cell of the true idleness field. Since the Watchman Route Problem is NP-hard on maps with obstacles [8], we use the standard cover-then-tour approximation: a greedy art-gallery set cover under omnidirectional visibility [27] followed by a 2-opt geodesic tour. For fairness, teammates within R repel the agent radially with weight $1 - d/R$.

b) Persistence: We evaluate whether GeMPa maintains low idleness across the environment and whether performance continues to improve as robots are added. Training uses idleness normalised to $[0, 1]$ so its contribution to the observation and reward stays on a consistent scale; for evaluation interpretability we report the underlying age in seconds since a cell was last seen, averaged over free cells and the episode ($\bar{I}$). Cell ages are initialized to the evaluation horizon, reset to zero when observed and otherwise increased at every time step without clipping. This metric captures both delayed initial coverage and repeated monitoring. We additionally report coverage success: the fraction of episodes reaching 90% cumulative coverage.

GeMPa achieves the lowest mean idleness at every evaluated team size, with a mean per-map reduction of 45.1% relative to the strongest baseline at each size (Table I(b)). This improvement is accompanied by higher coverage success than the baselines (Fig. 4), indicating fresher monitoring without sacrificing spatial reach.

	Trained			Out of Distribution	
	1	2	4	8	16
(a) Mean idleness $\bar{I}$					
GeMPa (ours)	304.3	161.9	94.3	50.8	25.8
MA-G-PPO	860.3	681.6	467.6	273.7	127.1
GCS	454.8	314.2	209.8	173.5	162.6
Watchman	376.4	282.2	225.4	215.5	235.2
(b) Mean per-map reduction $\Delta\bar{I}$ vs. strongest baseline (%)					
Hard synthetic (4)	28.3	59.6	69.5	82.1	71.3
Other synthetic (4)	19.4	50.3	63.3	66.0	65.0
Real maps (6)	6.9	19.8	20.7	43.8	51.9
All maps (14)	16.6	39.9	46.8	61.1	61.2

TABLE I

Clean-suite monitoring across team sizes. (a) Full-episode mean idleness, lower is better; (b) Mean per-map idleness reduction relative to the strongest baseline in that map and team size. Hard synthetic maps are Circular ring, Spokes, Half ring, and Rectangular ring; parentheses indicate map counts.

Training behaviour We first compare the methods during learning, under an identical budget of 25 million agent-steps and five seeds each (Fig. 5). GeMPa converges to a normalised rollout idleness of 0.213 ± 0.010 , 44% below MA-G-PPO. The two ablations separate here in different ways. Removing the idleness sectors raises rollout idleness to 0.265 ± 0.005 , 25% above the full model, so direct access to the staleness field improves the training objective itself. Replacing the GRU with a parameter-matched MLP is instead indistinguishable during training, at 0.211 ± 0.005 .

Held-out evaluation. We assess zero-shot generalization to 14 unseen maps and to teams of eight or sixteen robots, respectively two and four times larger than the maximum used during training. As shown in Table I(a), each doubling of the team lowers idleness by more than 40%, for a total reduction of 91.5% from one to sixteen robots and the largest single reduction, 49.2%, occurs between eight and sixteen,

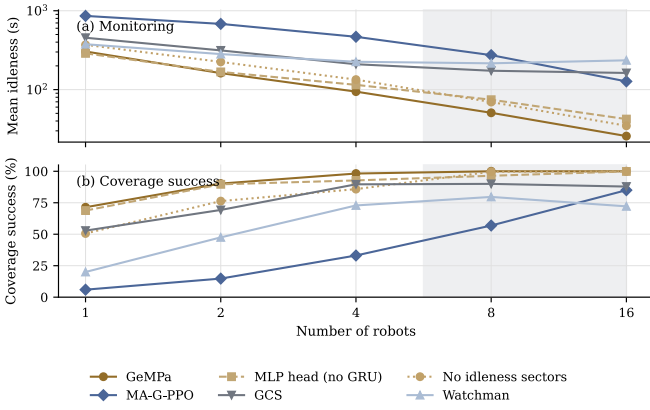


Fig. 4. GeMPa, ablations and baselines evaluated on the fourteen maps. (a) Full-episode mean idleness (b) Episodes reaching 90% cumulative coverage. Shading marks team sizes beyond training.

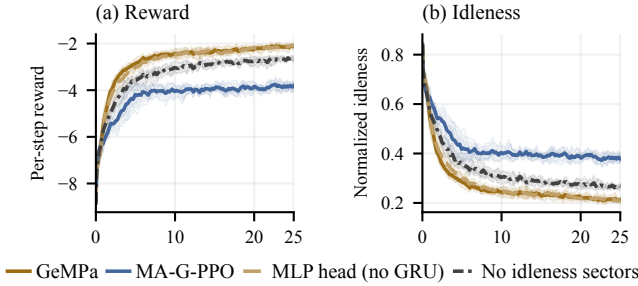


Fig. 5. Training reward and normalised rollout idleness over the 25 million training steps, with five seeds per method. Lines show the mean over seeds and shading one standard deviation. Reward is the trainer’s per-step diagnostic, not episode return.

so the scaling generalises beyond the training range. The baselines scale less steeply. The map-informed planners are at their best with a single robot, 47.1% and 56.3% below MA-G-PPO, but they gain least from a larger team: over the same sixteenfold increase they reduce idleness by only 64.2% and 37.5%, against 85.2% for MA-G-PPO, whose original evaluation already reported poor scaling across maps but good across teams [3]. Watchman even degrades by 9.1% between eight and sixteen robots. A route fixed in advance cannot send an added robot to what is currently stale. Coverage shows the same ordering: all GeMPa episodes reach 90% cumulative coverage at $N = 8$ and $N = 16$, whereas MA-G-PPO reaches it in 60.7% of trajectories at $N = 8$ and 85.8% at $N = 16$, while the deterministic planners saturate again at 90.0% and 87.9% for GCS and 79.6% and 72.1% for Watchman at $N = 8$ and $N = 16$ respectively. This generalization extends across all three evaluated layout types (Table I(b)), using maps unseen during training that differ in both layout and size. Averaged over the 14 maps, $\Delta \bar{I}$ reduction against the strongest baseline grows from 16.6% with one robot to 61.1% with eight. On the structured ring and spoke layouts the reduction reaches 82.1% at eight robots. On real floor plans it reaches 43.8% at eight robots and 51.9% at sixteen. Its weakest case, a single robot on real floor plans, still reduces idleness by 6.9%.

c) Adaptability: We ask whether monitoring survives changes and perturbations that occur mid-patrol. For the disturbance tests, three perturbations are introduced, none seen in training. Corridor closures add obstacles that seal rooms or force a new route, removal reduces N from $2 \rightarrow 1$, $4 \rightarrow 3$, $8 \rightarrow 6$ or $16 \rightarrow 12$, and teammate-information delays range from 1 to 8 s. Events occur near 400 s with $\pm 20\%$ jitter and persist to the end of the episode. For each rollout ℓ we measure $\Delta I_\ell = I_\ell(800 \text{ s}) - I_\ell(400 \text{ s})$. The 400 s reference is the training horizon, by which the policies are expected to have settled, and the jitter prevents a fixed onset from being exploited. The remaining 400 s gives the team the same horizon again to reach a new steady state, so ΔI_ℓ measures how far the disturbance shifts monitoring performance, with smaller being better.

Under disturbance, GeMPa shows the smallest ΔI_ℓ in every condition (Fig. 6). At sixteen robots the increase is 85% smaller than MA-G-PPO under corridor closure, 89% smaller under robot removal ($16 \rightarrow 12$, where MA-G-PPO is also the best baseline), and 76% smaller under the longest delay; averaged over the four removal pairs it is 64% smaller than the best baseline of each pair. Each disturbance meets a design choice that absorbs it. A closed corridor appears directly in the lidar returns as a change in proximity, whereas a predefined route encodes no notion of the closure at all: the agent keeps following a path that no longer exists and pays the cost until the route is replanned. A removed robot drops one token from the masked set and because the critic is conditioned on the active team size and the team is resampled every episode, the policy is optimised under changing composition rather than at a fixed size. A delayed update degrades gracefully because teammates are encoded by attention over relative distance and bearing, which stay approximately valid while the update is stale, and the recurrent state retains where they were last seen.

d) Unpredictability: We test whether improved monitoring comes at the cost of more regular revisit timing. Guo et al. [25] motivate evaluating efficiency and unpredictability jointly in graph-based patrols. In the visibility-based setting in which we operate, monitoring is credited when a location is observed, rather than only when a robot reaches it. We therefore evaluate revisit timing at grid cells, using global normalized revisit-gap entropy as a measure of temporal variability complementary to mean idleness, following [25]. The normalization bounds the score between zero and one. A cell has zero entropy when all recorded gaps fall in one bin; entropy increases as gaps spread more evenly across bins. Unlike mean idleness, this measures the diversity of revisit intervals rather than observation freshness. Averaging cell-level entropy values avoids confusing different regular schedules at different locations with irregular timing at an individual location. The score still depends on binning and observed events, so we interpret it alongside idleness and coverage rather than as a rate-independent measure. A deterministic sequence can also produce varied revisit intervals, so this metric does not establish that patrol trajectories are difficult to predict.

All four methods are compared on the same 14 maps. Fig. 7 shows that GeMPa retains revisit-time variability in a similar

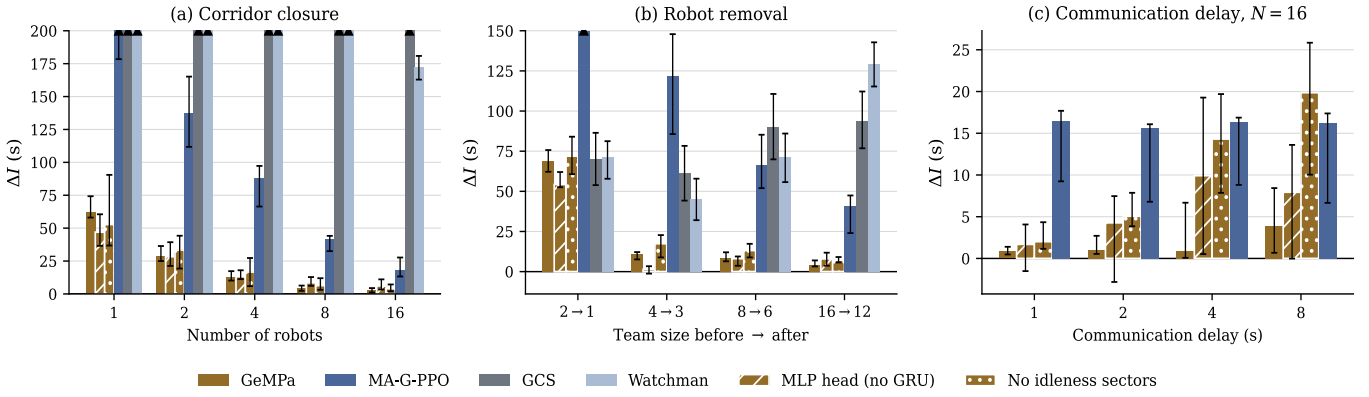


Fig. 6. Within-run idleness changes from 400 to 800s under (a) corridor closure, (b) robot removal, and (c) communication delay at sixteen robots. Positive values indicate increased idleness; vertical scales differ. Intervals are 95% seed-bootstrap intervals for learned methods and map-bootstrap intervals for planners. GCS and Watchman are omitted from (c) because neither uses the delayed teammate-information channel.

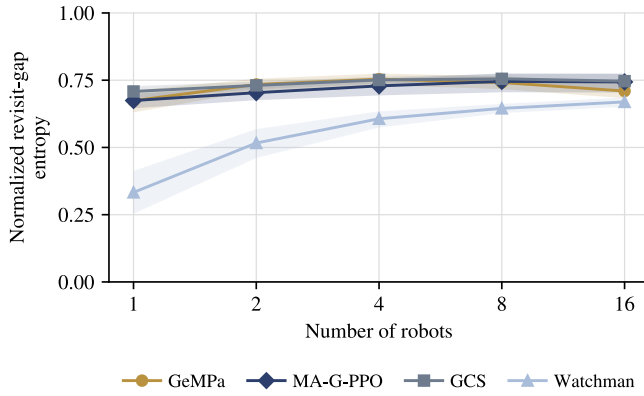


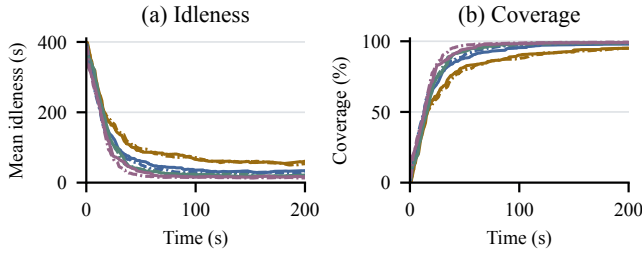
Fig. 7. Global normalized revisit-gap entropy on 14 clean maps, equally weighted; Bands show 95% bootstrap intervals over training seeds for learned methods and maps for planners.

range to MA-G-PPO while delivering substantially fresher monitoring (Fig. 4(a)). Its point estimates exceed Watchman’s for every multi-robot team size, and maintains a comparable entropy to MA-G-PPO and GCS.

e) Ablation study: We test whether temporal memory and idleness information in the observation space are necessary for repeated monitoring, replacing the GRU with a parameter-matched MLP and removing the idleness-sector inputs. The two ablations are already separated at training time: the sector-free variant converges to a lower reward, whereas the MLP reaches a similar reward and idleness (Fig. 5). This similarity does not carry over to held-out multi-robot monitoring: across all 14 maps, the MLP achieves similar mean idleness with one robot, but both ablations have higher mean idleness at every multi-robot team size (Fig. 4). The recurrence advantage grows with team size; at sixteen robots, removing the GRU increases idleness by approximately 64%, compared with 35% when removing idleness sectors. At four robots, coverage success falls by approximately 5 and 11 percentage points, respectively. These results support the contribution of memory and explicit idleness information to repeated monitoring, beyond their effect on initial coverage.

Communication delay further encourages using GRUs and idleness sectors (Fig. 6c): the MLP idleness increases more than GeMPa’s at every tested delay. At an 8s delay, its increase is 2.0 times larger, compared with 5.1 times larger without idleness sectors. These results suggest that memory and explicit staleness observations provide complementary support when teammate information is outdated. Both components also reduce closure and removal deltas at higher number of robots.

f) Real-world deployment: We deploy GeMPa at three physical sites: an outdoor industrial site, an outdoor parking lot (both shown in Fig. 1), and an indoor office, covering maps $8\times$, $1.5\times$ and $0.5\times$ larger than the training map area. Two robots patrol the outdoor sites and test coordination qualitatively at a scale and on layouts never seen in training. The hardware platform used is the Unitree Go2 quadruped, each unit carrying a 16-beam XT16 lidar, with GeMPa running on-board on the Jetson CPU. Robot poses are shared over Cyclone DDS; one robot maintains the team idleness map and broadcasts coverage timestamps, from which each robot builds teammate tokens and idleness sectors locally. From Fig. 1, we observe two distinct patrol strategies emerging from the map configuration and scale. At the larger industrial site, the robots partition the space, with their pose centroids (i.e., the time-weighted mean of each robot’s poses over a segment) staying well separated for the whole run (Fig. 1, left). In the smaller parking lot, the division is dynamic instead: the two macro-areas are exchanged across consecutive one-minute segments, centroids swapping sides as idleness falls (Fig. 1, right). No region is assigned to either robot, and both patterns follow from each agent conditioning on its teammate’s recent position. The office provides the quantitative comparison between simulation and the real world (Fig. 8): for each team size from one to four robots, we record three 300s runs and compare mean idleness and cumulative coverage against simulation, using the same checkpoint, map, and recorded initial poses. Hardware experiments closely matches simulation performance: coverage agrees to within 0.6 percentage points at every team size, reaching 99.1% with four robots against 99.4% in simulation, and the idleness gap narrows from 6.9s with one robot to



Number of robots: — 1 — 2 — 3 — 4 — Hardware — Simulation

Fig. 8. Office hardware deployments and matched simulation with one to four robots, averaged over three runs per team size. Curves are truncated at 200 s of the 300 s rollouts, after which all configurations have converged.

3.3 s with four. Idleness falls 3.7-fold from one to four robots on hardware, compared with 4.0-fold in simulation, so the team-scaling effect transfers rather than only the single-robot behaviour.

V. CONCLUSION

We presented GeMPa, a multi-agent reinforcement learning policy for visibility-based multi-robot patrolling that combines an egocentric idleness summary, attention over a variable-size team, and recurrent temporal reasoning, with no patrol graph or task-allocation planner. A single checkpoint transfers to every held-out map and team size without retraining, outperforming every baseline in all cases. For *persistence*, it attains the lowest mean idleness at every team size across fourteen held-out maps, with a mean per-map idleness reduction of 45.1% relative to the next best method, and idleness falls 11.8-fold from one to sixteen robots. For *adaptability*, under corridor closure, robot loss and communication delay the within-run increase is up to 89% smaller than the best baseline. GeMPa retains revisit-time variability comparable to MA-G-PPO and higher than Watchman, alongside substantially fresher monitoring. Ablations validate that the feed-forward variant matches the full model at one robot yet is worse at every larger team, so the gain is coordination rather than capacity, and removing the idleness sectors costs performance from training onward. The same checkpoint runs unmodified on physical quadrupeds at three sites, matching simulated coverage to within 0.6 points and preserving the scaling.

Limitations and Future work: Collision avoidance is learned rather than guaranteed, risking occasional contacts; an explicit safety filter on the policy output would enforce separation independently of the reward. The action space is planar with fixed velocity limits, so the policy transfers to any platform tracking forward, lateral, and yaw commands but does not exploit platform-specific capability. Finally, patrol effort is spread uniformly over free space, and our evaluation measures revisit-time variability rather than predictability to an observer; incorporating a spatial prior over intrusion likelihood, and evaluating against an adversary rather than a fixed metric, are the natural next steps.

ACKNOWLEDGMENTS

The authors wrote all text in this paper. AI tools were used solely for language editing and grammar checking of

author-written text and figure reformatting. The authors take full responsibility for the content.

REFERENCES

- [1] A. Machado *et al.*, “Multi-agent patrolling: An empirical analysis of alternative architectures,” in *Multi-Agent-Based Simulation II*. Springer, 2002, pp. 155–170.
- [2] Y. Chevaleyre, “Theoretical analysis of the multi-agent patrolling problem,” in *Proceedings of the IEEE/WIC/ACM International Conference on Intelligent Agent Technology*. IEEE Computer Society, 2004, pp. 302–308.
- [3] J. Chen *et al.*, “Multi-agent reinforcement learning for visibility-based persistent monitoring,” in *Proceedings IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2021, pp. 2563–2570.
- [4] Headquarters, Department of the Army, *FM 3-98: Reconnaissance and Security Operations*, 2023.
- [5] —, *FM 3-24.2: Tactics in Counterinsurgency*, 2009.
- [6] —, *ATP 3-21.8: Infantry Platoon and Squad*, 2016.
- [7] J. O’Rourke, *Art Gallery Theorems and Algorithms*. Oxford University Press, 1987.
- [8] W.-P. Chin and S. Ntafos, “Optimum watchman routes,” *Information Processing Letters*, vol. 28, no. 1, pp. 39–44, 1988.
- [9] D. Portugal and R. Rocha, “A survey on multi-robot patrolling algorithms,” in *Technological Innovation for Sustainability*, 2011, pp. 139–146.
- [10] Q. Guo and J. Peng, “Persistent monitoring for points of interest with different data update deadlines,” *Sensors*, vol. 24, no. 4, p. 1224, 2024.
- [11] M. Mishra *et al.*, “Multi-agent deep reinforcement learning for persistent monitoring with sensing, communication, and localization constraints,” *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 2831–2843, 2025.
- [12] C. Yu *et al.*, “The surprising effectiveness of PPO in cooperative multi-agent games,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [13] R. Lowe *et al.*, “Multi-agent actor-critic for mixed cooperative-competitive environments,” in *Advances in Neural Information Processing Systems*, 2017.
- [14] S. Carlsson and B. J. Nilsson, “Computing vision points in polygons,” *Algorithmica*, vol. 24, no. 1, pp. 50–75, 1999.
- [15] D. T. Lee and A. K. Lin, “Computational complexity of art gallery problems,” *IEEE Transactions on Information Theory*, vol. 32, no. 2, pp. 276–282, 1986.
- [16] S. Eidenbenz *et al.*, “Inapproximability results for guarding polygons and terrains,” *Algorithmica*, vol. 31, no. 1, pp. 79–113, 2001.
- [17] J. S. B. Mitchell, “Approximating watchman routes,” in *Proceedings ACM-SIAM Symposium on Discrete Algorithms*, 2013, pp. 844–855.
- [18] E. Packer, “Computing multiple watchman routes,” in *Proceedings International Workshop Experimental Algorithms*, 2008, pp. 114–128.
- [19] S. Ntafos, “Watchman routes under limited visibility,” *Computational Geometry*, vol. 1, no. 3, pp. 149–170, 1992.
- [20] C. G. Cassandras *et al.*, “An optimal control approach to the multi-agent persistent monitoring problem,” *IEEE Transactions on Automatic Control*, vol. 58, no. 4, pp. 947–961, 2013.
- [21] C. Song *et al.*, “Optimal control for multi-agent persistent monitoring,” *Automatica*, vol. 50, no. 6, pp. 1663–1668, 2014.
- [22] S. L. Smith *et al.*, “Persistent robotic tasks: Monitoring and sweeping in changing environments,” *IEEE Transactions on Robotics*, vol. 28, no. 2, pp. 410–426, 2012.
- [23] F. Pasqualetti *et al.*, “On cooperative patrolling: Optimal trajectories, complexity analysis, and approximation algorithms,” *IEEE Transactions on Robotics*, vol. 28, no. 3, pp. 592–606, 2012.
- [24] N. Rezazadeh *et al.*, “A sub-modular receding horizon approach to persistent monitoring for a group of mobile agents over an urban area,” *IFAC-PapersOnLine*, vol. 52, no. 20, pp. 217–222, 2019.
- [25] L. Guo *et al.*, “Balancing efficiency and unpredictability in multi-robot patrolling: A MARL-based approach,” in *Proceedings IEEE International Conference on Robotics and Automation*, 2023, pp. 3504–3509.
- [26] A. Jaegle *et al.*, “Perceiver: General perception with iterative attention,” in *Proceedings International Conference on Machine Learning*, vol. 139. PMLR, 18–24 Jul 2021, pp. 4651–4664.
- [27] S. K. Ghosh, “Approximation algorithms for art gallery problems in polygons,” *Discrete Applied Mathematics*, vol. 158, no. 6, pp. 718–722, 2010.